Teaching Notes

Companion to learning-objectives.md. Contains detailed teaching sequences, lease abstraction threads, CRE parallels, and worked examples organized by objective.

Objective 1: The Holy Trinity of Agentic Engineering — Teaching Notes

Where This Is Genuinely New

Where this is genuinely new: The deliberate switching between roles within a single AI workflow session. CRE professionals wear these hats over weeks or months on a deal; in agentic engineering, you switch hats within minutes. The speed of role-switching is a new skill.

Lease Abstraction Thread

Lease Abstraction Thread: The instructor maps the three roles to the lease abstraction plugin: *PM* — "What terms matter for this deal type? Office leases need different fields than industrial." *Engineer* — "How do I structure the extraction pipeline? Classification first, then targeted extraction." *Designer* — "How should the output look for the analyst who receives it?" Within one design session, you define what to extract (PM), design how to extract it (Engineer), then format the deliverable (Designer).

Objective 2: The Four Pillars — Teaching Notes

Where This Is Genuinely New

Where this is genuinely new: The Prompt pillar. Context assembly maps to due diligence, Model selection maps to consultant selection, Tools map to trades. But structured prompt design — crafting precise instructions that manage AI behavior through system prompts, constraints, and output formatting — has no clean CRE equivalent. This is a new craft.

Lease Abstraction Thread

Lease Abstraction Thread: The instructor diagnoses a bad lease abstract using the Four Pillars: *Context* — "I fed in the lease but not the Ontario Commercial Tenancies Act; the AI didn't flag a non-compliant clause." *Model* — "The model hallucinated a renewal option that doesn't exist; it lacks the capability to distinguish between standard and non-standard terms without reference material." *Prompt* — "I said 'summarize the lease' instead of 'extract the following 15 fields per the office lease template'; the AI produced a narrative summary instead of a structured abstract." *Tools* — "The lease was a scanned PDF; without OCR the AI was working from garbled text." Each failure has a specific pillar fix — a diagnostic framework students will apply to their own workflows.

Objective 3: Value-Delivery Fit (Paired VPD Lite + BMC Awareness) — Teaching Notes

Teaching Sequence

The Four Pillars ↔ Engineering Substrate Bridge (teaching sequence):

The instructor opens with a callback to Objective 2:

> *"The Four Pillars — Context, Model, Prompt, Tools — you learned those as a diagnostic framework: when output is wrong, which pillar failed? Now I'm going to show you the same four components from a different angle. They're not just diagnostic tools. They're the **engineering capability substrate** — the resources your system needs to deliver value. Context is your data and memory. Model and Prompt are your behavioral control plane. Tools are your action surface. Same components. In Obj 2 you asked 'what broke?' Now you're asking 'what do I need to deliver?'"*

Four Pillars (Obj 2)	Engineering VPD Substrate (Obj 3)	Diagnostic vs. Design
Context	Data / Context / Memory / State	"Was context insufficient?" → "What context does the system need?"
Model	Model capability	"Was the model wrong for this?" → "What model capability is required?"
Prompt	Prompt / Policy Stack	"Was the prompt ambiguous?" → "What behavioral controls are needed?"
Tools	Tooling / Action Surface	"Was a tool missing?" → "What tools must the system access?"

VPD Paired Exercise

VPD Paired Exercise:

Business side (Customer Profile → Value Map): - Customer jobs split by type: execution, decision, validation, coordination - Pains: trust/review burden, compliance exposure, cognitive load - Gains: confidence, control, auditability, throughput - Products & services (product form + autonomy mode) - Pain relievers (specific mechanisms, not slogans) - Gain creators (net new capabilities)

Engineering side (System Task Profile → Delivery Map): - System tasks (what system must do to support user jobs) - Failure pains (hallucination, timeout, stale data, false positives, cost spikes) - Performance gains needed (accuracy, latency, throughput, intervention rate) - Capabilities/components → the Four Pillars reframed as substrate - Failure relievers (schema validation, citation enforcement, HITL gates) - Performance creators (caching, batching, context pruning, model routing)

Cross-layer traceability (practiced on worksheet): - Rule 1: "If a business pain reliever has no engineering mechanism, it's marketing copy." - Rule 2: "If an engineering capability doesn't trace to a user job or pain, you're building the wrong thing." - Participants annotate trace lines and flag unmatched items.

CRE Parallels

CRE Parallels:

Concept	CRE Parallel
Business VPD	Tenant Needs Assessment — before you design the space, understand the occupier
Engineering VPD	Building Capability Assessment — can this building deliver what the tenant needs?
Cross-layer fit	Lease ↔ Building spec alignment — the lease promises a thing; the building must deliver it
BMC awareness	Back-of-envelope pro forma — does the deal math even work?
Customer jobs split	Lease renewal has all four: drafting (execution), deciding rate (decision), verifying terms (validation), routing approval (coordination)

Where This Is Genuinely New

Where this is genuinely new: The customer jobs split (execution/decision/validation/coordination) has no CRE equivalent — CRE professionals don't categorize their workflow tasks this way. Cross-layer traceability — demanding that every business claim map to an engineering mechanism — is a discipline from product engineering that most CRE professionals haven't encountered. The cost sniff test is familiar (back-of-envelope pro forma), but the full cost principle (tokens + human review + failure/rework + cost of delay) requires thinking about cost categories that don't exist in physical development.

Documentation Chain Note

Documentation Chain note: The VPD grounds the OFD. The paired VPD validates fit before the participant writes the objective function (Obj 4). This adds a new first link to the chain: VPD → OFD/PRD → ADR → DDD → TDD. The cross-layer traceability discipline from this objective connects forward to TDD (Obj 9) — acceptance criteria should trace not just to the OFD but through the OFD back to the VPD's validated user jobs and pains.

Lease Abstraction Thread

Lease Abstraction Thread: The instructor walks through a paired VPD for the lease abstraction plugin as the "I do" demonstration before students create their own:

Business VPD (Customer Profile → Value Map): - Jobs: Extract key terms (execution), determine if terms are market-standard (decision), verify no missing clauses (validation), route abstract to deal team (coordination) - Pains: Manual abstraction takes 2-4 hours per lease, inconsistent output across analysts, missed critical dates, junior analysts lack jurisdiction knowledge - Gains: Consistent structured output, jurisdiction-aware extraction, critical date flagging, analyst time freed for judgment tasks

Engineering VPD (System Task Profile → Delivery Map): - System tasks: Classify property type, load jurisdiction-specific legislation, extract structured fields, validate completeness against template - Failure pains: Wrong property type classification cascades to wrong template, hallucinated terms, missed jurisdiction-specific requirements, context overflow from large leases - Capabilities (substrate): Context (lease text + legislation + template + domain knowledge), Model (legal text comprehension), Prompt (structured extraction instructions per property type), Tools (PDF reader, structured output formatter)

Cross-layer trace: "The business pain 'missed critical dates' maps to the engineering mechanism 'critical date extraction with completeness validation.' The business gain 'jurisdiction-aware extraction' maps to the engineering capability 'dynamic legislation loading based on classification.' If a business claim has no engineering mechanism, it's marketing copy."

The instructor then flags: "Notice how the engineering VPD forced us to think about classification and dynamic context loading *before* we wrote a single line of specification. The VPD surfaced the architecture."

Objective 4: The 8-Step Process + OFD/PRD — Teaching Notes

8-Step ↔ Development Lifecycle (teaching dialogue)

The 8-Step Process is the development lifecycle applied to AI. The instructor walks through each step with its CRE parallel, using the lease abstraction plugin as the thread:

> *"You already know this process. Every development project follows the same arc. Let me show you how each step maps."*

Step	Agentic Engineering	CRE Development Lifecycle	Lease Abstraction Example
1. Discover	Identify the problem, the user, the opportunity	**Site Selection** — find the right opportunity before you design anything	"Who needs lease abstracts? Analysts drowning in 50-page leases. Pain: 2-4 hours each, inconsistent output, missed critical dates."
2. Plan	Define scope, constraints, success criteria (OFD/PRD)	**Feasibility** — can this work? What are the constraints? What does success look like?	"OFD: ≥95% field completeness, never fabricate terms, escalate unfamiliar clauses."
3. Design	Choose overall architecture — which components, which workflow shape	**Schematic Design** — massing, floor plates, general concept before any construction detail	"Classification-gated pipeline. Classify first, then load jurisdiction-specific context, then extract. Not one monolithic prompt."
4. Structure	Model the domain, define shared language (DDD)	**Construction Documents** — detailed specs that every trade reads the same way	"DDD: lease types, jurisdiction codes, template field definitions, domain vocabulary (CAM, TI, gross-up)."
5. Implement	Build the skills, agents, plugin components	**Construction** — build what the documents specify	"Write the classify-lease skill, the extract-sections skill, populate the knowledge directory."
6. Validate	Check outputs against OFD criteria	**Inspection** — does the work meet code and spec?	"Run the plugin on a test lease. Did it extract all 15 required fields? Did it identify the jurisdiction correctly?"
7. Test	Evaluate quality holistically (TDD/EDD)	**Commissioning** — does the whole system work end-to-end?	"Feed it leases from three different provinces. Do abstracts meet analyst standards? Where does quality degrade?"

Step	Agentic Engineering	CRE Development Lifecycle	Lease Abstraction Example
8. Document	Record what was built, decisions made, lessons learned	**As-Builts** — document what was actually constructed, not just what was designed	"Update the ADR with decisions made during build. Note which jurisdiction templates need expansion."

> *"If you skip Schematic Design and go straight to Construction Documents, you're drawing details for a building you haven't designed yet. Same thing happens with AI — people jump from 'I have a problem' straight to 'write me a prompt,' skipping steps 2-4 entirely. That's why their output wanders. The 8-step process prevents the wander."*

The key teaching insight: CRE professionals rarely skip steps in physical development — the consequences are too visible and expensive. But they skip steps routinely with AI because the consequences are invisible until the output arrives. The 8-step process makes the AI development process as disciplined as the physical one.

Teaching Sequence

OFD-first principle (teaching sequence): The objective function is taught *before* the PRD template. Participants answer three questions before writing a single requirement:

1. **Primary objective** — "What does success look like for this workflow?" (CRE parallel: "What's the target return on this deal?")
2. **Hard constraints** — "What can't be violated?" (CRE parallel: "What are the dealbreakers in underwriting?")
3. **Escalation triggers** — "When should the AI stop and hand off to you?" (CRE parallel: "When do you call the principal instead of making the call yourself?")

VPD input

VPD input: The objective function takes the paired VPD from Objective 3 as input. Your primary objective, hard constraints, and escalation triggers should trace directly to the user jobs and pains validated in the VPD.

Then — and only then — participants write the PRD to operationalize those objectives. The PRD answers "what to build" in service of the objective function's definition of "what

good looks like." At 201, the OFD fields remain physically embedded as the first section of the PRD document. At 301, the OFD separates into a standalone document with a full metric hierarchy, acceptance thresholds, and abstain/escalate policy.

Intent Engineering

Intent engineering connection (teaching sequence):

After participants complete the OFD fields, the instructor names what they just did:

> *"What you just wrote — a primary objective, hard constraints, escalation triggers — has a name in the industry. It's called intent engineering. Most people who use AI are still in era 1: prompt engineering — crafting individual instructions. The industry has moved to era 2: context engineering — managing what the AI knows (that's Objective 6). But the frontier — the thing that separates workflows that scale from tools that plateau — is era 3: intent engineering. You're encoding your purpose, your trade-off logic, and your decision boundaries into parameters the agent can execute. That's what the OFD is."*

The OFD fields as intent parameters:

OFD Field	Intent Parameter	What It Encodes
Primary objective	Trade-off logic	What to optimize for when competing priorities conflict ("maximize field completeness" vs. "minimize false positives")
Hard constraints	Decision boundaries	What the agent must never do regardless of context (absolute rules, not trade-offs)
Escalation triggers	Resolution hierarchies	When to stop acting and defer to human judgment (the frontier, written in advance)

The documentation chain as progressive specification:

The instructor draws the parallel to the full chain:

> *"The documentation chain — VPD → OFD → PRD → ADR → DDD → TDD — progressively translates abstract purpose into concrete, machine-actionable parameters. The VPD validates 'is this worth doing?' The OFD defines 'what does good look like?' The PRD operationalizes it. The ADR and DDD structure it. The TDD evaluates it. By the time you reach TDD, every abstract aspiration has been translated into a testable criterion. That's the progression — from intent to implementation to evaluation."*

The Microsoft Copilot lesson (motivating data point):

> *"Here's why this matters beyond your own workflow. Microsoft Copilot was adopted by 85% of the Fortune 500. But Gartner found only 5% moved to large-scale deployment. The failure wasn't the model — it was the Intent Gap. Organizations essentially hired 40,000 new digital employees without ever telling them how the company makes trade-offs. No OFD. No escalation triggers. No hard constraints. Just 'here's a smart tool, go.' Sound familiar? That's what happens every time someone opens ChatGPT and says 'summarize this lease' without defining what success looks like. The OFD prevents that — for your workflow, and eventually for your organization."*

"Seniority" in the AI era (teaching line):

> *"In the AI era, seniority isn't what you know in your head. It's your ability to externalize what you know into a framework an agent can execute. The person who can write a clear OFD with the right escalation triggers — because they've done 500 lease abstractions and know exactly where judgment is required — that person is more valuable with AI than without it. The person who's never done the work can't write the OFD. AI didn't change that equation. It widened it."*

This connects directly to the competence arbitrage framing from course positioning and reinforces the supervision problem.

Why OFD-first matters — the convergence principle

Why OFD-first matters — the convergence principle: In iterative AI development, the quality of your objective function determines the quality of your iterative convergence. When you feed a specification to an AI agent repeatedly (whether through manual refinement or automated loops), the agent builds incrementally on its own previous work. A weak objective function — or none at all — produces iterations that wander: each pass refines the build, but there's no external anchor for "is this getting

better?" A strong objective function produces iterations that converge: each pass can be measured against criteria that were defined before the agent started and that don't change between iterations.

The CRE parallel is immediate: your investment criteria (OFD) determine whether your underwriting process (PRD-driven workflow) converges on a good deal or wanders through bad ones. You define your criteria before you evaluate the opportunity — not after you've already fallen in love with the deal.

This connects forward to Frontier Recognition (Obj 8) — participants aren't just recognizing the frontier, they're writing it down in advance as part of their design discipline. It also connects forward to TDD (Obj 9) — the acceptance criteria participants write should trace directly back to the OFD fields defined here. If the acceptance criteria don't connect to the objective function, you're evaluating against criteria the process generated rather than criteria you defined up front.

Where This Is Genuinely New

Where this is genuinely new: The front-loading discipline. CRE professionals often start doing before planning — "let me just ask ChatGPT and see what happens." The 8-step process enforces Discover → Plan → Design before Implement, which requires more patience than most AI users exercise. The PRD template is familiar (pro forma), but writing one for an AI workflow requires thinking about machine capabilities and constraints rather than market conditions. The objective function is particularly new — in CRE, investment criteria are second nature, but most AI users never define what "good" means before they start prompting. The OFD-first discipline makes the investment criteria the starting point of the entire workflow design, not an afterthought.

Documentation Chain Note

Documentation Chain note: The chain description and per-document roles are in the instructor dialogue above (§Intent Engineering). The remaining documents are taught in Objectives 7 (ADR, DDD) and 9 (TDD).

Lease Abstraction Thread

Lease Abstraction Thread: The instructor demonstrates OFD-first discipline on the lease abstraction plugin:

OFD (defined before any requirements): 1. **Primary objective:** "Extract all key commercial terms from a lease into a structured abstract matching the property-type template, with ≥95% field completeness." 2. **Hard constraints:** "Never fabricate terms not present in the source document. Never apply residential tenancy rules to a commercial lease (or vice versa). Never skip critical dates (commencement, expiry, renewal option deadlines, rent escalation dates)." 3. **Escalation triggers:** "Unfamiliar clause types not in the template. Ambiguous terms requiring legal interpretation. Mixed-use properties that don't cleanly classify. Jurisdictions not in the knowledge base."

Then the PRD that serves the OFD: - Problem: Manual lease abstraction is slow, inconsistent, and error-prone - Users: Analysts, asset managers, deal teams - Requirements: Property type classification, jurisdiction detection, template-matched extraction, completeness validation - Success metrics: Field completeness rate, critical date capture rate, analyst time savings

The instructor highlights: "Notice I defined the escalation triggers *before* I wrote requirements. The triggers ARE the frontier boundary — they tell the AI when to stop and hand off. If I'd written the PRD first, I might have over-promised what the system can do."

Role Track: Alternative CRE Parallels for the 8-Step Process

For participants without deep construction/development experience (brokers, some analysts), the 8-step process also maps to the deal lifecycle:

Step	Development Lifecycle	Deal Lifecycle (alternative)
1. Discover	Site Selection	**Prospecting** — identify the opportunity
2. Plan	Feasibility	**Qualification** — can this deal close? What are the constraints?
3. Design	Schematic Design	**Deal Structuring** — term sheet, LOI, general deal shape
4. Structure	Construction Documents	**Due Diligence** — detailed investigation, shared data room
5. Implement	Construction	**Documentation** — draft the lease, negotiate terms
6. Validate	Inspection	**Legal Review** — does documentation match deal terms?
7. Test	Commissioning	**Execution** — sign, fund, does everything work?
8. Document	As-Builts	**Closing File** — record what was actually agreed, lessons learned

The instructor should use the pre-course survey data to gauge the room's development exposure. If the audience skews toward brokers and analysts, lead with the deal lifecycle and use development as the reinforcing example. If the audience skews toward asset managers and development professionals, lead with the development lifecycle and mention the deal parallel briefly.

Both parallels deliver the same pedagogical payload: you already follow a structured multi-step process in your CRE work; agentic engineering is the same discipline applied to AI.

Objective 5: Task Decomposition — Teaching Notes

Where This Is Genuinely New

Where this is genuinely new: The decomposition heuristics are different for AI. In construction, you decompose by trade specialty (electrical, plumbing, framing). In AI, you

decompose by context boundaries (what information does each step need?), token limits (how much can the AI hold in working memory?), and validation gates (where must a human check before proceeding?). These constraints have no CRE equivalent.

Lease Abstraction Thread

Lease Abstraction Thread: The instructor decomposes the lease abstraction into delegable units with explicit boundary rationale:

```
1. Classify lease (property type + jurisdiction)     ← AI, low risk
   Context boundary: needs only first 2-3 pages
   Human checkpoint: CONFIRM classification before proceeding

2. Load relevant context                              ← System (deterministic)
   Based on classification: legislation + template + domain knowledge
   No AI judgment needed — routing logic

3. Extract sections per template                      ← AI, medium risk
   Context boundary: lease text + loaded context
   Token budget: lease (~30K) + legislation (~15K distilled) + template (~2K)

4. Validate completeness against template             ← AI + human review
   Context boundary: extracted fields + template checklist
   Human checkpoint: REVIEW any flagged gaps or uncertain extractions

5. Format and deliver                                 ← System (deterministic)
   No AI judgment — template formatting
```

The instructor points out the decomposition heuristics that differ from CRE: "In construction, you decompose by trade. Here, I decomposed by *context boundary* — each step gets only the information it needs. Step 1 needs just the first pages. Step 3 needs the full lease plus loaded context. If I gave Step 1 the entire 50-page lease plus legislation, it would be overwhelmed for a simple classification task. Context boundaries are the new trade boundaries."

Objective 6: Context Assembly — Teaching Notes

Where This Is Genuinely New

Where this is genuinely new: The challenge isn't gathering information (CRE professionals do due diligence daily). It's deciding what information the AI needs vs. what's noise. A 50-page lease is too much context; the AI loses focus. The key terms and deal-specific constraints are sufficient. Learning to compress domain knowledge into effective AI context is a new skill that maps loosely to due diligence but requires different judgment about relevance and granularity.

Lease Abstraction Thread

Lease Abstraction Thread: The instructor uses the lease abstraction plugin to teach context assembly as the most critical engineering challenge — and the one most likely to be underestimated.

The problem (demonstrated live): A typical commercial lease is 30-50 pages (~30K-50K tokens). The Ontario Commercial Tenancies Act adds another ~40-60K tokens raw. A lease abstract template is ~2K tokens. Domain knowledge (property type characteristics, market conventions) adds ~5-10K tokens. Total raw context: 80-120K tokens — approaching or exceeding practical working limits even for large-context models, because the model needs reasoning room beyond the raw input.

Side-by-side demonstration: - **Thin context:** "Summarize the key terms of this lease." → AI produces a generic narrative, misses jurisdiction-specific terms, uses wrong template format - **Assembled context:** Classified as Ontario office lease → loads distilled Ontario Commercial Tenancies Act (key sections only, ~8K tokens), office lease template (~2K), office domain knowledge (~3K), plus the lease text (~35K) → total ~48K tokens with room for reasoning → AI produces a structured abstract matching the template with jurisdiction-aware annotations

Five strategies for context overflow (taught as engineering techniques, not workarounds):

1. **Knowledge distillation** — Pre-process reference materials into concentrated forms. The full Ontario Commercial Tenancies Act is ~50K tokens; a distilled version covering the sections relevant to lease abstraction is ~8K tokens. The skill's `knowledge/` directory contains pre-distilled legislation, not raw statutes. CRE parallel: you don't bring every bylaw to a site inspection — you bring the relevant code sections.

2. **Multi-pass architecture** — Don't process the entire lease in one pass. First pass: classify and extract the table of contents / section headers. Second pass: extract fields from each section with targeted context. Third pass: validate completeness and cross-reference. Each pass uses a smaller, focused context window. CRE parallel: due diligence is phased (Phase I, Phase II, Phase III), not monolithic.

3. **Dynamic context loading** — Load only the context relevant to the current classification. An Ontario office lease loads the Ontario Commercial Tenancies Act + office template. A BC residential lease loads the BC Residential Tenancy Act + residential template. The classification step (which needs minimal context) determines what the extraction step loads. CRE parallel: you don't order an ALTA survey for a property in Ontario — you load the right reference for the jurisdiction.

4. **Chunked document processing** — Split the lease into logical sections (premises, rent, maintenance, insurance, renewal, termination) and process each with section-specific context and instructions. Reassemble the structured outputs. CRE parallel: a building inspection is done system by system (HVAC, electrical, plumbing), not all at once.

5. **Context budgeting** — Assign a token budget to each context component and enforce it. Lease text: 35K max. Legislation: 10K max (distilled). Template: 3K. Domain knowledge: 5K. Reasoning room: 15K minimum. Total budget: 68K. If a lease exceeds the text budget, chunk it (Strategy 4). If legislation exceeds its budget, distill further (Strategy 1). CRE parallel: a development pro forma has line-item budgets; you don't let one cost category consume the entire budget.

The instructor emphasizes: "Context assembly isn't just 'give the AI more information.' It's *engineering decisions about what to load, when to load it, and how much room to leave.* These are the same resource allocation decisions you make on a construction project — except the resource is tokens, not dollars."

Teaching Notes

Teaching note: This is where skills-as-knowledge-containers is introduced. The plugin's skills don't just contain instructions — they contain (or reference) the domain knowledge needed for the task. A skill for Ontario office leases references the distilled Ontario Commercial Tenancies Act. This is dynamic context loading implemented through the plugin architecture students are learning.

Objective 7: Claude Code Fundamentals + ADR/DDD — Teaching Notes

Documentation Chain Note

Documentation Chain note: The ADR (Document 2) and DDD (Document 3) are taught through worked examples embedded in the lab, not as standalone exercises. Participants *interpret* these documents rather than *producing* them — appropriate for a 200-level course where the goal is chain literacy. Production of ADRs and DDDs is a 300-level skill.

Where This Is Genuinely New

Where this is genuinely new: Everything in this objective. Claude Code components have CRE analogies (agents = subcontractors, skills = spec sections), but the analogies are scaffolding, not substance. Knowing when to use an agent vs. a skill vs. a slash command is a genuinely new competency with no direct CRE equivalent. The focus is on recognition and appropriate selection — "I need a subcontractor for this" vs. "I need a standard detail for this" — not deep implementation. The plugin concept is introduced here as the *container* that bundles components into a distributable package — think of it as the project itself (not one trade or one spec section, but the whole coordinated build). Participants see a minimal plugin structure so they understand the capstone target before they start building toward it.

Teaching Notes

Component hierarchy (teaching note):

```
Plugin (the distributable package — like the project itself)
├── Agents (autonomous specialists — subcontractors)
├── Skills (reusable knowledge — spec sections)
├── Slash Commands (user-invocable shortcuts — work orders)
└── [optional: hooks, MCP servers — not taught at 201]
```

At 201, participants learn to *use* all four component types and *understand* how plugins bundle them. The capstone (Obj 10) has participants *build* a minimal plugin with at least one skill. At 301, participants build production plugins with multiple agents, skills, and hooks.

Lease Abstraction Thread

Lease Abstraction Thread: The structured lab uses the lease abstraction plugin as the guided exercise. Participants work with a pre-built plugin that includes: - `plugin.json` manifest defining "lease-abstraction" with description and component list - `classify-lease` skill — identifies property type and jurisdiction from the first pages - `extract-sections` skill — extracts structured fields using the loaded template - `knowledge/` directory — contains distilled legislation and templates per property type/jurisdiction

Participants use all four component types: invoke the plugin (plugin), trigger the classification agent (agent), read the extraction skill (skill), and run the `/abstract` command (slash command). The ADR they annotate explains: "Why classification-gated loading instead of loading everything? Because context budgeting (Obj 6) requires targeted loading — the classification step needs minimal context, while the extraction step needs lease + legislation + template. Loading everything into every step wastes tokens and degrades quality." The DDD they annotate defines: lease types (office, industrial, residential, mixed-use), jurisdiction codes, template field mappings, and the domain vocabulary (CAM charges, TI allowances, gross-up provisions, etc.).

Productive failure exercise (participatory)

Productive failure exercise (participatory — everyone builds wrong together): Before showing the structured lab, the instructor AND all participants type the same poorly-specified prompt into Claude Code on their own machines: "Create a skill that abstracts leases." Every participant gets a different (but similarly flawed) output — this LLM variance itself becomes a teaching moment.

Common failure patterns across the room: generic templates instead of property-type-specific ones, ignored jurisdiction (Ontario assumptions applied to a BC lease), raw statute loaded instead of distilled sections (consuming the context window), missed critical dates because no one told the AI they were critical. Some outputs will hallucinate renewal options, others will use residential templates for commercial leases, others will produce narrative summaries instead of structured abstracts.

The instructor pauses the room: "Look around. Everyone typed the same thing. Did anyone get the same result?" Class discussion surfaces the variation — demonstrating that the problem isn't randomness, it's the absence of specification. Every participant's AI made different wrong decisions because no one told it the right ones.

The instructor then names what went wrong: "I skipped the PM hat — I didn't classify the lease type first. I skipped the VPD — I didn't ask whether the system can actually deliver jurisdiction-aware extraction. I skipped the OFD — I didn't define what 'good' looks like. I let Claude make every design decision I should have made. By the time I saw the finished skill, the architecture was already wrong. *Context without intent is a loaded weapon with no target.* I gave Claude Code plenty of context — the lease, the statute — but zero intent. No objective function. No constraints. No escalation triggers. The result? Confidently wrong output in perfect formatting."

Then the class rebuilds together — the instructor walks through building the same skill with the OFD from Objective 4, the decomposition from Objective 5, and the context assembly strategy from Objective 6, with participants following along on their own machines. The same tool produces a correct, jurisdiction-aware abstract when the human externalizes intent before the tool starts filling gaps. Participants can compare their rebuilt outputs side-by-side — the convergence toward correct output (vs. the divergent failure outputs) demonstrates the power of specification.

This reinforces every prior objective's discipline at the exact moment participants are tempted to abandon it — when they first get their hands on a powerful tool that feels like it can figure things out on its own. It also previews the capstone challenge: if one skill goes wrong without a design, imagine an entire plugin.

The Butterfly Effect in Agentic Workflows (teaching note)

After the productive failure exercise, the instructor names the error propagation pattern:

> *"What you just saw is the Butterfly Effect in agentic workflows. A small error at step 1 — wrong classification — cascaded through every subsequent step. Wrong classification loaded wrong legislation, which produced wrong extractions, which missed critical dates. Each step looked reasonable in isolation. By the time you see the final output, the original error is invisible — buried under layers of confident, well-formatted work. And unlike a human junior who might hesitate or ask a question when something feels off, the AI charges forward at full speed. This is why we decompose with human checkpoints (Obj 5), define success criteria before the agent starts (Obj 4), and practice frontier recognition (Obj 8). The checkpoints interrupt the cascade. The OFD gives you something to evaluate against. Without both, a small mistake at the top propagates silently through the entire workflow."*

This connects forward to Obj 8 (Diagnosing Failures) — participants will learn to trace failures back to their origin point in the cascade, using the Four Pillars to identify which component introduced the original error. It also reinforces the Supervision Problem from course positioning: the faster and more autonomous the agent, the more critical your ability to validate its work at each checkpoint.

Objective 8: Diagnosing Failures & Frontier Recognition — Teaching Notes

Where This Is Genuinely New

Where this is genuinely new: Pillar diagnosis has a loose CRE parallel (troubleshooting a construction deficiency), but the specific failure modes are AI-native: context insufficiency, model capability limits, prompt ambiguity, tool gaps. Frontier recognition maps to highest-and-best-use analysis, but the AI frontier is moving — what's impossible today may be routine in 6 months. Learning to assess a moving frontier is a skill CRE professionals don't practice in their domain, where zoning and building code boundaries are relatively stable.

Frontier Recognition connects to the trust/verification overlay from the VPD (Obj 3) — the escalation triggers participants defined in the OFD trace back to the trust requirements validated in the VPD.

Lease Abstraction Thread

Lease Abstraction Thread: The three pre-generated failure examples in Part A use lease abstraction outputs:

1. **Context failure:** The AI was given a BC lease but loaded the Ontario Commercial Tenancies Act. The abstract references Ontario-specific provisions that don't exist in BC. Fix: the classification step misidentified the jurisdiction → fix the classification skill's jurisdiction detection.
2. **Prompt failure:** The AI extracted residential lease fields (security deposit, pet policy) from a commercial lease. The template was wrong — an office lease needs CAM reconciliation, escalation structure, and TI allowances. Fix: the prompt loaded the wrong property-type template → fix the routing logic between classification and extraction.

3. **Model failure:** The AI fabricated a renewal option clause, citing "standard commercial practice," when the lease has no renewal option. The model filled a template field with hallucinated content rather than marking it absent. Fix: add a hard constraint to the OFD — "Never fabricate terms not in the source document; mark absent fields as 'Not Present in Lease'" → trace back to the OFD's hard constraints.

Part B frontier recognition uses lease abstraction tasks: "AI can extract structured fields" (automatable) → "AI can flag a non-standard indemnification clause" (AI-assisted, human review) → "AI can determine whether a tenant's exclusive use clause conflicts with another tenant's permitted use" (human judgment — requires understanding business relationships, not just text).

Objective 9: Iterative Refinement, Quality Judgment & TDD — Teaching Notes

Teaching Notes

TDD vs. EDD — two kinds of acceptance criteria (teaching note):

Not all acceptance criteria work the same way. At the 201 level, participants write both kinds without needing to formally categorize them. But the instructor should name the distinction:

- **TDD criteria are deterministic** — the answer is binary. "The lease abstract includes all critical dates" is either true or false. "The rent roll totals match the GL" passes or fails. This is like **building inspection**: the wiring meets code or it doesn't. Same input, same expected output, every time.
- **EDD criteria are probabilistic** — the answer is judgment-based, measured against a rubric or threshold. "The market analysis captures the key trends" is a matter of degree. "The tenant communication is professional and accurate" requires evaluation against a quality standard, not a binary check. This is like **appraisal**: the value is a range, subject to methodology and professional judgment.

At the 201 level, the discipline is the same for both: *define what "good" looks like before the AI produces anything.* The criteria-writing skill is what matters. At the 301 level, the distinction becomes operational:

	TDD (Deterministic)	EDD (Evaluation-Driven Development)
Target	Code/data behavior	Probabilistic task-level quality
Pass/fail	Binary, reproducible	Threshold-based, statistical
CRE parallel	Building inspection	Appraisal
Artifact	Unit/integration tests	Eval suites (datasets + rubrics + graders)
Use for	Parsers, validators, schema checks, data transforms	Prompt quality, retrieval accuracy, task success rates, hallucination rates
201 level	Write binary acceptance criteria	Write rubric-based acceptance criteria
301 level	Build deterministic test harness	Build evaluation harness (Evals Spec)

How they fit together

How they fit together: TDD protects correctness of the machinery (did the data parse correctly?). EDD protects correctness and usefulness of the behavior (did the workflow produce a good result?). Both run in CI at production scale. At the 201 level, participants write both kinds of criteria on paper. At 301, they build both kinds of harnesses.

Economics check (BMC callback)

Economics check (BMC callback): The cost sniff test from Objective 3 adds an economics dimension to iterative refinement: is the refinement effort worth the outcome improvement? This prevents infinite-loop polishing where each iteration costs more than the quality gain is worth.

Alignment Drift Awareness (301 preview)

Alignment drift (teaching note — seed for 301):

After the refinement exercise, the instructor plants a forward-looking concept:

> *"The acceptance criteria you just wrote aren't disposable. In production, they don't validate once — they monitor continuously. An agent that passes your TDD criteria on day 1 may drift by day 30 as data changes, edge cases accumulate, or the model updates. This is called alignment drift — the agent's behavior gradually diverges from your original intent. At the 301 level, you build feedback loops that detect and correct this drift in real-time. For now, know that the criteria-writing discipline you're learning here has a second life as ongoing alignment monitoring."*

This connects to the iterative refinement mindset: quality judgment isn't a one-time event, it's a continuous discipline.

MCP Awareness (301 preview)

MCP awareness (teaching note — seed for 301):

> *"The context infrastructure you've been building — skills-as-knowledge-containers, dynamic context loading, distilled legislation in the `knowledge/` directory — connects to a broader industry standard called MCP (Model Context Protocol). MCP standardizes how agents access data across organizations. It was donated to the Linux Foundation in December 2025 and has nearly 100 million monthly SDK downloads. Claude Code can connect to MCP servers for live data access — your plugin's `knowledge/` directory is a static version of what MCP delivers dynamically. That's 301 content. For now, the patterns you've learned — what to load, when to load it, how much room to leave — apply regardless of how the data arrives."*

Documentation Chain Note

Documentation Chain note: TDD is taught here because it connects directly to quality judgment — and because it closes the loop back to the OFD. The acceptance criteria participants write in this exercise should trace directly to the OFD fields they defined in Objective 4. This is the critical connection: the OFD's "what does success look like?" becomes the TDD's "did we achieve it?" If acceptance criteria don't connect to the objective function, you're evaluating against criteria that emerged during the build rather than criteria you defined before it started.

This completes the documentation chain: VPD (Obj 3, produced) → OFD/PRD (Obj 4, produced) → ADR/DDD (Obj 7, interpreted) → TDD (Obj 9, produced). The OFD and TDD

bookend the chain as independent anchors — the OFD defines the target, the TDD evaluates against it. This independence is what makes iterative refinement convergent rather than circular. At 301, the TDD position in the chain splits: TDD for deterministic components, Evals Spec (the EDD artifact) for probabilistic/agentic components.

Where This Is Genuinely New

Where this is genuinely new: Writing acceptance criteria *before* seeing AI output requires a different mental discipline than reviewing output *after* it's produced. CRE professionals are accustomed to reviewing work product reactively (the appraisal lands on your desk, you review it). TDD inverts this: you define what "passing" looks like first, then evaluate against your own criteria. The criteria-writing skill transfers from underwriting (define go/no-go before you fall in love with the deal), but the discipline of writing them *for AI output* before prompting is new. The TDD/EDD distinction adds another layer: some of your criteria will be binary (like building code), and some will be rubric-based (like an appraisal). Knowing which is which helps you evaluate AI output more honestly.

Lease Abstraction Thread

Lease Abstraction Thread: The instructor demonstrates acceptance criteria writing using the lease abstraction plugin, showing both TDD and EDD criteria:

TDD criteria (deterministic — binary pass/fail): - "All critical dates extracted: commencement, expiry, renewal deadline, rent escalation dates" — either present or missing - "Rent structure matches source: base rent amount, escalation schedule, CAM charges" — numbers match or they don't - "Jurisdiction correctly identified" — Ontario or not Ontario - "No fabricated terms: every extracted field traces to a specific lease section" — verifiable against source

EDD criteria (probabilistic — rubric-based): - "Clause summaries capture material terms without omitting significant conditions" — judgment call, measured against a rubric (complete / partially complete / insufficient) - "Abstract is organized in a format useful to an analyst reviewing the deal" — quality standard, not binary - "Non-standard clauses are flagged with appropriate context" — requires judgment about what's "standard"

The instructor highlights: "Notice that your TDD criteria trace directly back to the OFD's hard constraints — 'never fabricate terms' became a binary acceptance test. Your EDD criteria map to the OFD's primary objective — 'extract key terms matching the template'

requires judgment about completeness and usefulness. The OFD wrote your acceptance criteria for you."

Economics check: "I've run three refinement cycles on the clause summaries. Each cycle improved the EDD score by ~5%. But each cycle costs 50K tokens (~$0.15) and 3 minutes of review time. Is the fourth cycle worth it? The cost sniff test from Obj 3 says: compare marginal improvement against marginal cost, including your review time."

Objective 10: Capstone — CRE Workflow Plugin — Teaching Notes

Why a plugin, not just a skill or a design doc

Why a plugin, not just a skill or a design doc: The plugin is the natural capstone because it forces participants to exercise every layer of the documentation chain: - **VPD** → Is this workflow worth building? For whom? Can we deliver? - **OFD** → What should this plugin do? What does success look like? When should it stop? - **PRD** → What skills, agents, or commands does it need? What are the requirements? - **ADR** → Which components for which tasks? Why this structure? - **DDD** → The domain model expressed as skill prompts and plugin manifest fields - **TDD** → Test with `--plugin-dir`, validate outputs against OFD criteria

A skill is one component. A design document is paper. A plugin is the integrated system — and building one requires integrating everything they've learned. The minimum viable plugin needs only a name and one skill, so the floor is achievable; the ceiling (multiple skills, agents, hooks) rewards participants who work faster.

Bloom's level note

Bloom's level note: The capstone is classified as Apply/Create rather than pure Create. In 90 minutes with material learned 24 hours ago, participants will primarily operate at the Apply level — following templates and the plugin scaffold to assemble their plugin — with Create-level moments where they make original decisions about their specific CRE workflow and skill design. This is a realistic and valuable outcome. Pure Create-level performance (designing a production plugin without scaffolding) is a 300-level skill that develops through sustained practice after the course.

Lease Abstraction Thread

Lease Abstraction Thread — Capstone Framing: By this point, participants have watched the lease abstraction plugin develop across every objective:

Objective	What participants witnessed
Level 0	Single-turn vs. multi-step quality gap on lease abstraction
Obj 1	PM/Engineer/Designer role-switching applied to lease abstraction
Obj 2	Four Pillars diagnosis of a bad lease abstract
Obj 3	Paired VPD validating fit for the lease abstraction plugin
Obj 4	OFD/PRD defining success criteria and requirements
Obj 5	Task decomposition with context boundaries and human checkpoints
Obj 6	Context assembly strategies for 50-page leases + statutes
Obj 7	Hands-on with the plugin structure; productive failure without design
Obj 8	Diagnosing jurisdiction/template/hallucination failures; frontier mapping
Obj 9	TDD/EDD acceptance criteria with economics check

The capstone instruction becomes: "You've seen every concept in this course applied to one plugin — lease abstraction. You've seen the VPD, the OFD, the decomposition, the context strategy, the component structure, the failure modes, and the acceptance criteria. Now build your own. Pick a CRE workflow you do repeatedly. Apply the same pipeline. You have the pattern; the capstone asks you to replicate it for *your* domain."

This framing reduces capstone anxiety (they've seen it done) while maintaining challenge (they must adapt the pattern to a new domain). The Worked Example Effect predicts stronger capstone performance than if concepts were taught in pure abstraction.

Course Closing — Teaching Notes

Reverse Transfer Exercise (5 min)

Before the next-steps and reinforcement plan, the instructor facilitates a brief reflection:

> *"I want you to think about something you might not expect. Every discipline you learned this weekend — OFD, acceptance criteria, task decomposition with checkpoints, context assembly, diagnostic frameworks — was taught for AI workflow management. But here's the thing: most of you don't apply this rigor to managing human teams either."*

Individual reflection (2 min):

> *"Name one practice from this course that you're going to apply to a human team or human deliverable in the next two weeks. Not an AI workflow — a human one. Maybe it's writing an OFD before you delegate the next project. Maybe it's writing acceptance criteria before you review an analyst's work. Maybe it's decomposing an assignment into checkpointed stages instead of 'get back to me Friday.'"*

Pair share (2 min): Participants share with a neighbor. The instructor invites 2-3 volunteers to share with the room (1 min).

Why this matters pedagogically: This exercise accomplishes three things: 1. **Validates the management-skills framing.** Participants articulate in their own words that agentic engineering IS management. 2. **Creates a second application domain.** Participants now have two contexts to practice these disciplines — AI workflows and human teams — doubling the likelihood of retention and transfer. 3. **Handles the AI-skeptic gracefully.** If a participant leaves unconvinced about AI's role in their daily work, they still leave with a concrete management improvement. The course delivered value regardless.